TESTING

CSE 362

Software Development and Professional Practice

Winter 2024

© Royal Military College of Canada

Suppose …

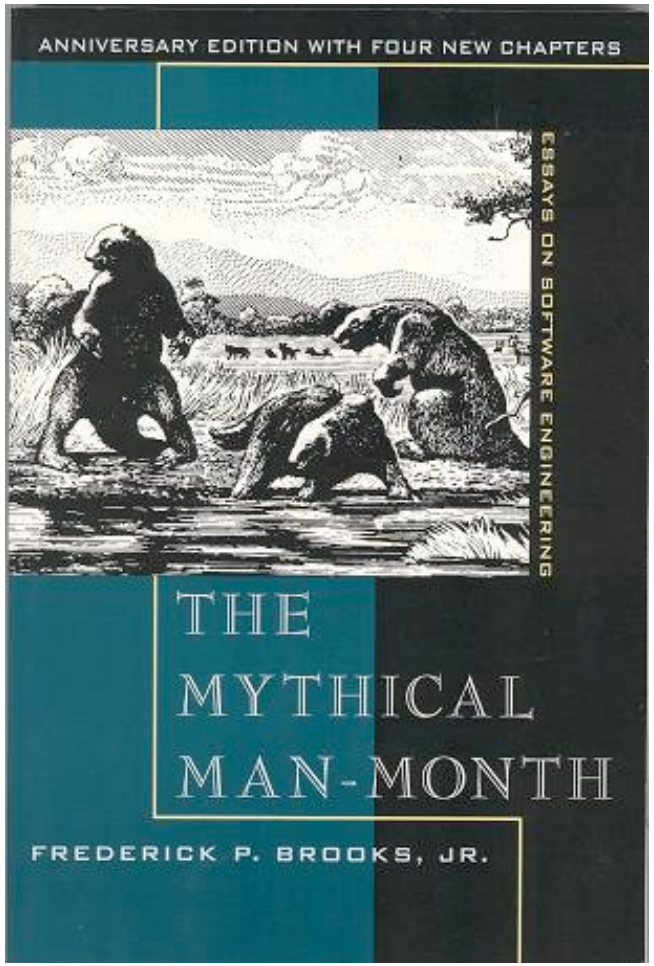
- You have a software project that needs completing.
- Your local software guru says it would take him about 12 months to get it done – 12 "man-months".
- You know your competitor will have a similar product on the shelves in 5 months.
- If you hire another 5 more coders (from "Rent-A-Geek"), can you get the project done in 2 months?

The Issues

- Sub-division of the overall task will take extra time.
- Not all programmers will have equal skill levels – some extra training will be required.
- Working as a team will necessitate extra time for communications!
- But you still have to develop large software projects as a team – the current demand for reduced time to market requires it.

“The Mythical Man-Month”

- by Fred Brooks (Addison-Wesley, 1995)
- Fred was the project manager for IBM System/360 and OS/360.
- The first real treatment of this subject – a classic.



“Brook’s Law”

“Adding manpower to a late software project makes it later.”

- He likens the best software team to a surgical team:
  - One clear leader: the surgeon.
  - With just enough supporting people to make the surgery possible.

- The teams should be as small and “tight” as possible.

Applied to Agile Teams

- Same thing?
- If an agile team is running late on a project, would you add more members to the team and expect to see an improvement?

Applied to Agile Teams, Cont.

Some would say that an XP ("Extreme Programming") team can overcome Brook's Law:

- Integrate new coders by pairing them up with experienced programmers – pair programming.
- The experienced programmers are still producing at the same time as training the newbies.
- Continue to use all the other tools that facilitate rapid development: TDD, Repositories, etc.

Reminder – Extreme Programming

- Design and code at the same time.
- Keep it simple.
- prototyping, refactoring.
- Pair programming.
- Automated unit testing.
- TDD ("Test Driven Development")
- User acceptance testing.

TDD

The three Laws of TDD:

1. You may not write production code until you have written a failing unit test.
2. You may not write more of a unit test than what is sufficient to cause a failure, and not compiling is a failure.
3. You may not write more production code than what is sufficient to pass the current failing test.

TDD, Cont.

- So, tests should be written at the same time as the production code. You are always one test ahead of what the production code can pass.
- Tests are used to express your best understanding of what your code is supposed to do.
- As the amount of production code grows so do the number of tests.
- Testing code should be in separate files from production code.

XP Testing

- Every coder is responsible for building and running their own unit tests on their code.
- Your code goes in the repository after it has passed all of your tests.
- Daily! Before the nightly, automated build.
- Does this mean that the entire system will work as planned?

Two Quotes on Testing

- Kent Beck: "*Optimism is the occupational hazard of programming; testing is the treatment.*"

- Brian Marick: "*The first mistake people make is thinking that the testing team is responsible for assuring quality.*"

Testing

- Testing alone cannot prove conclusively that code is without errors.
- Don't rely on testing to make good code out of bad – write the best code from the best models first!
- Testers should not be beginners! Testing requires knowledge of the entire system and its design, along with knowledge and experience with the testing techniques.

System Failure

- Often the result of a combination of events that is nearly impossible to predict or test for.

- A few examples ranging from just annoying to deadly.

- Classical examples of system failures:
  - Arianne 5 Rocket: overflow problem
  - Patriot Missiles: rounding problem

Phoenix Pay System for the Canadian Government

- IBM was the only company to bid on the creation of a new payroll system in 2011. The contract went to 2019. It is likely worth about $185 million to IBM, due to amendments covering IBM's assistance in fixing the current problems. The original contract was only for $5.7 million.

- IBM was instructed to base the system on PeopleSoft, by Oracle.

- <u>The government was to be responsible for installation, testing and training</u>. They wanted to save money and do it themselves.

Phoenix Pay System, Cont.

- First installed in February 2016.

- There was no proof that the system was working completely, but a $4.8 million award was paid to 340 executives in Public Services and Procurement Canada who installed the system ahead of its scheduled delivery date.

- In April, 2016 the existing 40 year old payroll system was taken off line after the second release of Phoenix was installed with minimal and inadequate testing.

Phoenix Pay System, Cont.

- About eighty thousand government employees have been overpaid or underpaid. And about 700 have not been paid at all!

- This is not the first time that IBM has had problems getting such large systems working. (See the Queensland Health Payroll problems from 2010, for example).

- Problems have been linked to inadequate testing, not enough training and a lack of understanding about how long it would take employees to master the new system.

Phoenix Pay System, Cont.

- The system has also been responsible for 11 widespread internal data breaches of employee data.
- When released, the problems resulted in 7,000 calls per day to a help system that was designed to handle a maximum of 2,200 calls per day.
- In July, 2016, there were 82,000 outstanding employee reported problems with the system.
- It is estimated that by the time another system is installed, it will have cost $2.2 billion to keep Phoenix somewhat running.

Microsoft Zune 30 Meltdown

- Apparently 2006 models of this MP3 player just froze up at midnight, Dec. 31, 2008.

http://slashdot.org/article.pl?sid=09/01/04/2034248

- Eventually Microsoft acknowledged the problem.

- Here is the offending C code:
- Can you spot the problem?

```
#define ORIGINYEAR 1980
BOOL ConvertDays(UINT32 days, SYSTEMTIME* lpTime) { int dayofweek,
  month, year;
  UINT8 *month_tab;
  //Calculate current day of the week
  dayofweek = GetDayOfWeek(days);
  year = ORIGINYEAR;
  while (days > 365) {
    if (IsLeapYear(year)) {
      if (days > 366) {
        days -= 366;
        year += 1;
      }
    }
    else {
      days -= 365;
      year += 1;
    }
  }
…
```

- Intent of the code: repeatedly decrement the days count—by 365 in common years and by 366 in leap years—and each time we do that, we'll increment the year number by 1.
- At the end of the process, the year count should have reached the correct current value.

Unit testing
Integration and system testing.

The F.I.R.S.T Rules

- **F**ast tests are more likely to be run as often as they should be.
- **I**ndependent tests will not depend on each other. You should be able to run tests in any order.
- **R**epeatable tests can be run in any hardware/OS environment.
- **S**elf-validating tests either pass or fail.
- **T**imely tests follow the three laws of TDD.

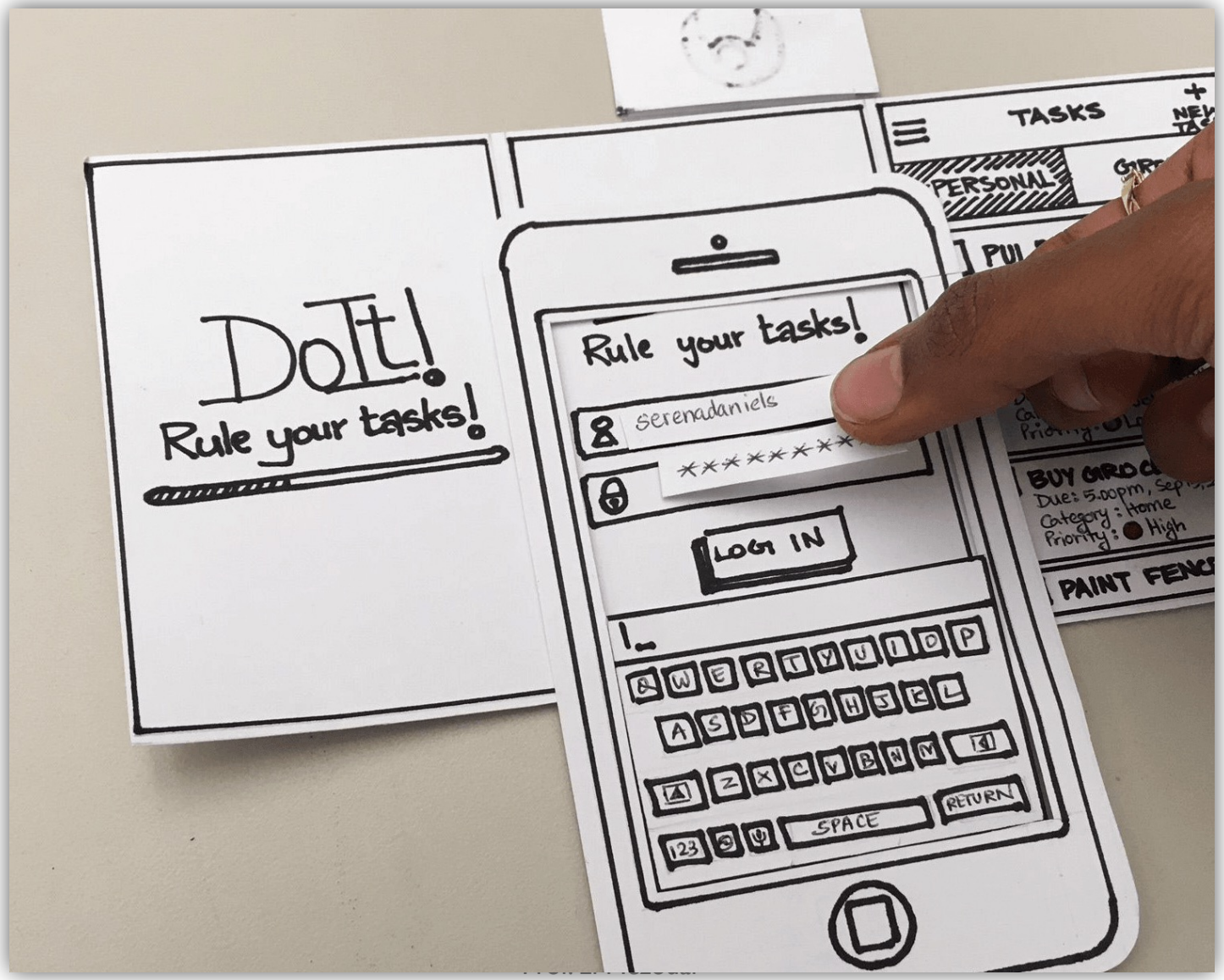
Overview of the Testing Process

Process	**Testing**
Coding	Unit Testing
Design	Integration Testing
Requirements Analysis	Validation Testing
System Engineering	System Testing

Overview, Cont.

- Test Planning – Allocate and schedule resources. Do this early on – as soon as models stabilize.
- Usability Testing – Test user interface. (See "Paper Prototyping", for example.)
- Unit Testing – Test individual methods and classes.
- Integration Testing –
  - First separate classes by threads.
  - Start by identifying independent classes and test them first, then test dependent classes, layer by layer.
  - Cluster testing – test a bunch of collaborating classes together.
  - Use regression testing, stub and driver code as needed (more about these later!).

Aside – Paper Prototyping



Overview, Cont.

- System Testing – Test all the classes together.
  - –Validation Testing tests the requirements listed in the RAD and user manual.
  - – Performance Testing checks non-functional requirements and additional design goals from the SDD. (Done by developers).
  - – Acceptance and Installation Testing – Check the system against any agreements made. (Done by clients with help from developers, if needed).

More Definitions…

- A Component is the class, the cluster of classes, the package or one or more subsystems that is being tested.
- A Test Case is a set of inputs and expected results that exercises a component with the purpose of causing failures and detecting faults.
- A Test Stub is partial implementation of a component upon which the component being tested depends.
- A Test Driver is a partial component that depends on the component being tested.
  - Stubs and Drivers allow the component to be *harnessed* – isolated from the rest of the system.

More Definitions…

- A Correction is a change to a component made to repair a fault. *A correction can introduce new faults…*

- A system that fails after a correction has *regressed*.

- Carrying out the previous set of integration tests again is called Regression Testing.

Unit Testing

Unit Testing

- Common types of Unit Testing:
  - Equivalence testing
  - Boundary testing
  - Path testing
  - State-based testing

Equivalence Testing

- A Blackbox technique.– (Deal with the input/output of the component, leave the internal behaviour alone. For now…)
- Minimal number of test cases.

- First, determine a set of equivalence classes, i.e. form groups of test inputs of similar behaviour or nature.
- You assume that the component behaves the same for all members of the equivalence class.

Equivalence Testing, Cont.

- Criteria for equivalence classes:
  - *Coverage* – every possible input would belong to one of the equivalence classes.
  - *Disjointedness* – no input belongs to more than one equivalence class.
  - *Representation* – if one member of an equivalence class causes an erroneous state, then the same state can be caused by any other member of that class.

- For each class, select at least two test inputs – a valid input and an input that should exercise your exception handlers.

Equivalence Testing, Cont.

- Example – suppose you are testing a class that stores dates, as given by a year and month.
- You would need six equivalence classes:
  - 31 day months, non-leap years
  - 31 day months, leap years
  - 30 day months, non-leap years
  - 30 day months, leap years
  - 28 day month, non-leap years
  - 29 day month, leap year
- Minimum of 12 tests!

Aside: Leap Year or Not

How to determine whether a year is a leap year

To determine whether a year is a leap year, follow these steps:

1. If the year is evenly divisible by 4, go to step 2. Otherwise, go to step 5.
2. If the year is evenly divisible by 100, go to step 3. Otherwise, go to step 4.
3. If the year is evenly divisible by 400, go to step 4. Otherwise, go to step 5.
4. The year is a leap year (it has 366 days).
5. The year is not a leap year (it has 365 days).

Equivalence Classes

- **Non-Leap Years Not Divisible by 4**:
  - All years that are not divisible by 4 fall into this class. They are common years with 365 days.
  - Example: 2019, 2021.
- **Leap Years Divisible by 4 but Not by 100**:
  - These years are divisible by 4 but not by 100. They are considered leap years with 366 days.
  - Example: 2024, 2028.
- **Common Years Divisible by 100 but Not by 400**:
  - These years are divisible by 100 but not by 400. Despite being divisible by 4, these are not leap years and have 365 days.
  - Example: 1900, 2100.
- **Leap Years Divisible by 400**:
  - These years are divisible by 4, by 100, and by 400. They are leap years with 366 days.
  - Example: 2000, 2400.

Boundary Testing

- Look at the "edges" of the equivalence classes.

- For example:

  – **Non-Leap Years Not Divisible by 4**:

  Boundary values are years just before and after multiples of 4.

  Test with years like 2018 and 2019 (just before 2020) and 2022 (just after 2021).

Equivalence and Boundary Testing

- The problem with these techniques is that they do not explore many combinations of input data.

- Often a strange combination will cause an error, and this will take many more tests to locate.

Path Testing

- A Whitebox technique.
  - (Focus on the internal structure of the component. Test every state of the component and every possible interaction of objects.)
- Exercise every possible path through the code at least once.
  - Exercise every branch of every if statement.
  - Run each loop – can you cause a loop not to run?
  - Throw each exception.

- A code coverage tool (*like the one in Eclipse*) can help you to confirm that you have tested everything at least once.

Path Testing, Cont.

- The problem with this technique is that it cannot show where code is missing – it only tests existing code.

State-based Testing

- Developed for GUI systems.
- A newer technique that is not fully fleshed out. (*And, the tools can be expensive!*)

- Based on the idea that objects (like a button) can have different states under different conditions (like active, inactive, pressed, mouseover, etc.)

- See Wikipedia entry on "List of GUI Testing Tools".

Integration Testing

Integration Testing

- Carried out after unit testing is complete.

- Start with the smallest combinations of components possible, and then let the groups get larger as testing proceeds.

- Still need drivers and stub code. (Note that Extreme Programming, or "XP", requires that all drivers are written before components are developed!).

Integration Testing, Cont.

- Four strategies:
  - Big Bang testing
  - Bottom-Up testing
  - Top-Down testing
  - Sandwich testing

- (*Who thinks up these names…*)

Big Bang Testing

- Once unit testing is complete, test full system.

- You don't need any more drivers, but it is very hard to pinpoint the source of an error.

- (*This is what you are going to do in your projects, isn't it?...*)

Bottom-Up Testing

- Start at the lowest layer (independent classes).
- Combine classes one at a time – double testing, triple testing, quadruple testing, etc.
- No stub code is required, only drivers to simulate the level above the one being tested.

- Sounds nice and safe to me!

Bottom-Up Testing, Cont.

- Interface faults are easily found.

- The problem with Bottom-Up is that UI components are tested last. A correction in this important top layer may require changes all the way back down…

Top-Down Testing

- Opposite to Bottom-Up (*that's a surprise!*)
- Uses stub code for the lower layers, as required. No drivers required.

- Starts with the UI – the most important part of the system.

Sandwich Testing

- Combination of bottom-up and top-down.
- Break the system down into three layers (to form the sandwich). Top, Bottom and Middle.
- Apply both bottom-up and top-down at the same time.
- You will not need drivers and stubs for the outer layers.

- "Modified Sandwich Testing": Test the layers independently and then test all together.

System Testing

Sandwich Testing

- Test the entire system to make sure in complies with the functional and non-functional requirements.

- Carry out:
  - Functional testing
  - Performance testing
  - Pilot testing
  - Acceptance testing
  - Installation testing

System Testing, Cont.

- Functional testing:
  - Also called requirements testing.
  - Compare the system to the functional requirements.
  - Blackbox technique.
  - Tests are extracted from the use case models.
  - The tester should choose the tests that are of the most importance to the user and the most likely to demonstrate a failure.
  - Test cases are created as they were with equivalence and boundary testing.

System Testing, Cont.

- Performance testing:
  - Examine the difference between the system and the design goals.
  - Extract tests from the SDD or the RAD.
  - Examples:
    - Stress testing
    - Volume testing
    - Security testing
    - Timing tests
    - Recovery tests

System Testing, Cont.

- Pilot testing:
  - See "Alpha Testing"
- Acceptance testing:
  - The client prepares benchmark tests.
  - You might test the new system vs. the legacy system.
- Installation testing:
  - The testing of the system after installation.
  - Probably repeat many of the earlier tests.
  - (Can't take too long!)

Post Release Testing

- This brief discussion above was all about pre-release testing.

- Alpha Testing is carried out by a small team of potential customers on site.

- Beta Testing is all the rage these days. Release to the public, but on a somewhat restricted basis. Must have some way of getting feedback from the beta testers in return for getting free software ahead of other people.

Corrections

- What to do when you find a fault?
- How do you reduce the odds of introducing new faults into the repaired component?
- (A big topic!)

- Techniques that can be used:
  - Problem tracking
  - Configuration management
  - Regression testing
  - Rationale maintenance

Some Measures of Software Testing

- Bugs found per tester per unit time (day/week/month).
- Total bugs found in a release.
- Total bugs found in a module or feature.
- Bugs found or fixed per build.
- Number of customer reported bugs.
- *And a lot more …*